

ARQUITECTURA DE CAPACIDADES

Tenda

Capacidades funcionales y herramientas tecnológicas

Propósito: Base para arquitectura general, vistas y plan de desarrollo por etapas

Alcance: Inventario, ventas, despacho, agente de inventario y capacidades transversales

Fuentes: Tenda-inventario.bpmn, Tenda-Ventas.bpmn y Tenda-Despacho.bpmn

Fecha: 17 de agosto de 2026

1. Alcance y criterios

Este documento convierte los BPMN en capacidades implementables y las vincula con componentes tecnológicos. No define todavía las vistas ni el backlog detallado; ambos deben derivarse después de estas capacidades y de las decisiones abiertas.

Principio rector: Tenda debe ser simple en la interfaz, no simple en la consistencia de datos. La experiencia puede ocultar complejidad, pero stock, pagos, webhooks y cambios de estado necesitan reglas explícitas, idempotencia y auditoría.

1.1 Decisiones que deben fijarse antes del desarrollo

- GraphQL será la API de dominio para web y mobile. OAuth/OIDC callbacks, webhooks de pago y correo, health checks y cargas firmadas a R2 seguirán siendo endpoints HTTP específicos; forzarlos dentro de GraphQL sería una mala decisión.
- El stock no será solo un campo quantity editable. La fuente de verdad será un libro de movimientos (StockMovement) y un saldo materializado o calculado; toda corrección dejará trazabilidad.
- Antes de enviar al cliente al pago se creará una reserva de stock con expiración. El pago aprobado confirma la salida; pago fallido, cancelado o vencido libera la reserva.
- Celery y Redis no son alternativas. Celery ejecuta tareas; Redis puede actuar como broker, caché, coordinador de locks y backend temporal. Los estados de negocio durables permanecen en PostgreSQL.

- MCP no reemplaza al orquestador del agente ni a la base vectorial. Se justifica solo como frontera estándar de herramientas; un agente interno puede llamar directamente a la capa de servicios de Django.
- El ambiente Dev no debe compartir base de datos, volúmenes, claves, bucket/prefijos ni red con Producción. Si ambos viven inicialmente en la misma VPS, se acepta como compromiso temporal, con dominios, proyectos Compose, redes y límites de recursos separados.
- LinkedIn debe ser secundario frente a correo y Google. Exige habilitar el producto OIDC y añade soporte con valor dudoso para el público objetivo; incluirlo no justifica convertirlo en el camino principal de registro.

1.2 Entidades de dominio mínimas

Dominio	Entidades principales	Regla de diseño
Identidad	User, Organisation, Membership, Role	tenant_id/organisation_id obligatorio en datos de negocio; autorización en cada servicio y resolver.
Inventario	Product, Variant/SKU, Category, MediaAsset, StockMovement, StockBalance	No borrar productos con historia; archivar. Cada ajuste crea movimiento.
Ventas	Order, OrderItem, StockReservation, Payment, PaymentProof, WebhookEvent	Estado transaccional explícito e idempotencia por operación externa.
Despacho	Shipment, ShipmentEvent, DeliveryConfirmation, Ticket, TicketMessage	Una máquina de estados única; historial inmutable de eventos.
Transversal	Notification, OutboxEvent, AuditEvent, IdempotencyKey	Efectos externos después del commit; reintentos seguros.
Agente	AgentRun, AgentSuggestion, Embedding, ModelEvaluation, UserFeedback	La IA propone borradores; no altera stock ni productos sin confirmación humana.

2. Mapa de procesos y etapas

Etapas	Proceso	Resultado funcional	Dependencias clave
1 • MVP	Inventario	Productos y stock confiables; carga manual, actualización, importación y consulta.	Identidad, PostgreSQL, R2, GraphQL, web/mobile, auditoría.

Etapa	Proceso	Resultado funcional	Dependencias clave
2 · MVP	Ventas	Pedido y cobro por pago online, depósito/transferencia o efectivo; conciliación con stock.	Reservas, proveedor de pago, Brevo, R2, Celery/Redis.
3 · MVP	Despacho	Preparación, seguimiento, confirmación de entrega y consultas/tickets.	Máquinas de estado, recordatorios, enlaces firmados, notificaciones.
4 · Upgrade	Agente de inventario	Creación asistida desde imagen, búsqueda y sugerencia de parámetros con validación humana.	NVIDIA VLM, PGVector, evaluaciones; MCP solo si aporta interoperabilidad.

3. Proceso de inventario · Etapa 1

El BPMN contempla actualización de existentes, carga manual, importación y carga por imagen. La capacidad base debe quedar completa sin depender del agente de la etapa 4.

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
INV-01	Entrada y dashboard	Al abrir la app, cargar resumen de productos, stock total, alertas, movimientos recientes y accesos rápidos. Respuesta paginada y parcial; no bloquear por datos secundarios.	Next.js / Expo + Apollo; Strawberry GraphQL; PostgreSQL
INV-02	Catálogo y consulta	Listar, buscar, filtrar y ordenar productos por nombre, SKU, categoría, estado y bajo stock. Cursor pagination; filtros aislados por organisation_id.	PostgreSQL (índices/trigram opcional); GraphQL
INV-03	Maestro de producto	Crear producto y, cuando corresponda, variantes/SKU. Campos base: nombre, SKU, categoría, unidad, precio, costo opcional, stock mínimo, estado e imágenes. Parámetros extensibles sin convertir todo a JSON.	Django models + PostgreSQL; validación Pydantic/Strawberry inputs
INV-04	Carga manual	Formulario corto con guardado de borrador, validaciones comprensibles y valores por defecto. Confirmar antes de crear stock inicial.	React Hook Form + Zod; mutations GraphQL
INV-05	Actualizar producto existente	Seleccionar producto y registrar entrada, salida, merma o corrección con cantidad, motivo y nota. Nunca sobrescribir el saldo sin movimiento asociado.	Servicio de inventario Django; transacción PostgreSQL
INV-06	Fotos y media	Tomar/subir imagen, validar tipo/tamaño, generar clave por tenant y cargar directo con URL PUT firmada. Guardar metadatos solo tras confirmar la carga.	Cloudflare R2 S3 API + boto3; Pillow; URLs firmadas
INV-07	Importación masiva	Aceptar CSV/XLSX; mapear columnas, mostrar previsualización, validar por fila y ejecutar como job. Entregar reporte descargable de aceptados y errores; reintento idempotente.	pandas/openpyxl; Celery + Redis; R2 para archivo/reporte
INV-08	Libro de stock	Registrar StockMovement inmutable con tipo, cantidad firmada, referencia (pedido/importación/ajuste), actor y timestamp. StockBalance se actualiza en la misma transacción.	PostgreSQL constraints + select_for_update; servicio Django
INV-09	Consistencia concurrente	Evitar saldo negativo y dobles operaciones con idempotency_key, locks de fila y constraints. Cada mutation crítica acepta clientMutationId/idempotencyKey.	PostgreSQL; Redis locks solo para coordinación no transaccional
INV-10	Alertas de stock	Detectar stock bajo o agotado y mostrarlo en dashboard; notificación configurable. No enviar una alerta repetida por cada consulta.	Celery; Redis deduplicación; Brevo/push opcional

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
INV-11	Auditoría y corrección	Mostrar historial por producto; permitir corrección compensatoria con motivo, nunca editar/eliminar movimientos contabilizados.	AuditEvent + StockMovement; Django Admin/soporte
INV-12	Exportación	Exportar inventario y movimientos filtrados a CSV/XLSX como tarea; enlace temporal al resultado.	Celery; openpyxl; R2 URL GET firmada
INV-13	Ciclo de vida	Archivar/reactivar productos. Impedir borrado si existen movimientos, pedidos o referencias; preservar imágenes según retención.	Django soft archive; lifecycle rules R2
INV-14	Modo sin IA	Todo flujo de alta debe funcionar manualmente aunque el servicio de IA esté caído, sin cupo o deshabilitado.	Feature flags/configuración; formularios manuales

4. Proceso de ventas · Etapa 2

El BPMN converge pago online, depósito y efectivo antes de descontar inventario. Se conserva esa convergencia, pero se agrega reserva previa, expiración y manejo de fallos para evitar sobreventa.

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
VEN-01	Crear pedido	El vendedor selecciona producto/variante, cantidad, precio efectivo, método de pago y modalidad de entrega. Capturar snapshot de nombre/precio para que cambios posteriores no alteren el pedido.	Order/OrderItem; mutation GraphQL transaccional
VEN-02	Disponibilidad y reserva	Validar stock y crear StockReservation con expiración antes de checkout. Operación atómica; impedir que dos compradores reserven la misma última unidad.	PostgreSQL row locks/constraints; Celery Beat para expiración
VEN-03	Ficha pública	Generar URL pública de producto/pedido, responsive y accesible. Exponer solo datos necesarios; nunca IDs secuenciales ni información del vendedor no autorizada.	Next.js SSR/ISR; token opaco firmado; CDN Cloudflare
VEN-04	Compartir enlace	Copiar/compartir por Web Share API o redes y enviar por email. Registrar canal, no contenido personal innecesario.	Web Share API/deep links; Brevo
VEN-05	Confirmación del cliente	Mostrar producto, cantidad, precio, reserva y vencimiento; botón comprar con protección contra doble clic/reenvío.	Next.js; idempotency key; Turnstile adaptativo

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
VEN-06	Datos del comprador	Recolectar contacto y despacho; separar datos de envío de datos tributarios. Minimizar, validar y definir retención. No afirmar que se emite boleta/factura sin una integración DTE.	GraphQL public mutation; cifrado en tránsito; políticas de retención
VEN-07	Abstracción de pagos	Definir PaymentProvider adapter: connectSeller, createCheckout, getStatus, refund y verifyWebhook. Credenciales por vendedor cifradas y revocables.	Django payments service; secret storage
VEN-08	Pago online	Crear preferencia/checkout usando credenciales del vendedor; redirigir al proveedor. El retorno del navegador no aprueba el pago: confirmar por API/webhook.	Mercado Pago Checkout Pro + OAuth (recomendado MVP)
VEN-09	Depósito/ transferencia	Crear enlace firmado para subir comprobante; validar tipo/tamaño; estado awaiting_review. El archivo no equivale a pago aprobado.	R2 URL firmada; Turnstile/rate limit; PaymentProof
VEN-10	Efectivo y pago manual	Permitir al vendedor registrar cobro presencial con monto, fecha y nota. Exigir confirmación explícita y auditar al actor.	Mutation GraphQL; AuditEvent
VEN-11	Webhooks idempotentes	Verificar autenticidad, persistir evento bruto, deduplicar por provider/event_id y responder rápido. Procesar después; reconsultar al proveedor si hay duda.	Endpoint HTTP Django; Outbox/Celery; WebhookEvent
VEN-12	Validar comprobante	Vendedor aprueba o rechaza con motivo; notificar al comprador. Rechazo no descuenta stock y puede liberar o extender reserva según regla.	GraphQL + Brevo; máquina de estados
VEN-13	Estado de pedido/pago	Estados mínimos: draft, reserved, awaiting_payment, payment_pending, paid, cancelled, expired, refunded. Transiciones centralizadas; no strings libres en resolvers.	Servicio explícito con enums/guards (recomendado); django-fsm-2 opcional
VEN-14	Conciliación de stock	Al aprobar pago, convertir reserva en salida de stock dentro de una transacción. Si falla, marcar reconciliation_required y alertar; nunca ocultar inconsistencia.	PostgreSQL transaction + StockMovement; Celery retry
VEN-15	Notificaciones	Enviar confirmación, cambio de estado, rechazo y vencimiento desde un outbox posterior al commit. Registrar delivered/bounced cuando Brevo informe el evento.	OutboxEvent; Celery; Brevo API/webhooks
VEN-16	Cancelación/ reembolso	Liberar reserva al vencer/cancelar. Para pago confirmado, registrar reembolso y movimiento compensatorio solo cuando corresponda; mantener historial.	PaymentProvider.refund; Celery; StockMovement
VEN-17	Etiqueta de despacho	Generar PDF/imagen con pedido, destinatario y datos de entrega; no confundir etiqueta interna con guía de un transportista.	WeasyPrint (recomendado) o ReportLab; R2

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
VEN-18	Conciliación y reportes	Vista/export de ventas por método/estado; detectar pagos sin pedido, pedidos pagados sin salida y eventos pendientes.	Consultas PostgreSQL; Celery exports; dashboard

5. Proceso de despacho · Etapa 3

El BPMN incluye cambio de estado, confirmación de recepción y ciclo de consultas. Los temporizadores aparecen sin duración y ligados a tareas instantáneas; la implementación debe usar agenda durable y reglas configurables.

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
DES-01	Inicio de despacho	Crear Shipment solo desde pedido pagado/confirmado y con etiqueta disponible. Copiar datos de envío como snapshot controlado.	Evento OrderPaid/Confirmed; Outbox + Celery
DES-02	Máquina de estados	Estados mínimos: pending, preparing, dispatched, delivery_check, delivered, issue, returned, cancelled, closed. Transiciones con actor, fecha y comentario.	Shipment/ShipmentEvent; servicio Django
DES-03	Preparación y envío	Vendedor registra transportista opcional, referencia/seguimiento, fecha y estado. Permitir operación manual aunque no exista API de carrier.	Mutations GraphQL; validación de estados
DES-04	Notificar cambios	Notificar al comprador cuando se prepara, despacha, entrega o surge incidencia. Dedupe por shipment + transition.	Brevo; push opcional; Celery
DES-05	Seguimiento programado	Crear recordatorios persistentes con due_at y política configurable. No usar countdowns largos en Redis como única fuente de verdad.	FollowUpSchedule en PostgreSQL; Celery Beat
DES-06	Confirmación del comprador	Enviar enlace opaco, firmado y expirable. Respuesta sí/no en endpoint público con rate limit y, si hay riesgo, Turnstile; una sola respuesta efectiva por solicitud.	Django HTTP endpoint; signed token; Turnstile
DES-07	Confirmar entrega	Respuesta sí registra delivered y cierra el despacho según política; conservar evidencia temporal y fuente de confirmación.	ShipmentEvent + AuditEvent
DES-08	Crear consulta/ticket	Respuesta no permite abrir o reutilizar ticket del envío. Número legible, categoría, pregunta inicial y estado open.	Ticket service; GraphQL/HTTP público
DES-09	Conversación	Comprador y vendedor intercambian mensajes asociados al ticket; notificar sin incluir datos sensibles completos en el correo.	TicketMessage; Brevo; URLs firmadas

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
DES-10	Resolución	Estados open, awaiting_seller, awaiting_buyer, resolved, closed. Permitir nueva consulta mientras el despacho esté en ventana de soporte; registrar resolución.	Máquina de estados + auditoría
DES-11	Recordatorios y expiración	Cadencias por configurar para consulta de entrega, espera de respuesta y autocierre. Escalar al vendedor antes de cerrar por tiempo.	Celery Beat; reglas en PostgreSQL
DES-12	Devolución/incidencia	Registrar devolución, pérdida, rechazo u otro resultado. Una devolución de stock requiere inspección/confirmación; no reponer automáticamente por una respuesta del comprador.	ReturnCase + StockMovement compensatorio
DES-13	Integración con carrier	Diseñar adapter opcional para cotizar, crear envío, obtener tracking y webhook. Fuera del MVP mientras no se seleccione proveedor.	CarrierProvider interface; implementación futura

6. Agente de inventario • Etapa 4

La etapa 4 es un upgrade sobre un inventario ya funcional. Su valor está en reducir digitación y detectar productos existentes, no en reemplazar las reglas transaccionales del backend.

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
AGE-01	Entrada visual	Recibir una o más fotos desde cámara/galería, con compresión razonable y metadatos mínimos. Nunca enviar al modelo una URL pública permanente.	Expo/Next.js; R2 URL firmada
AGE-02	Comprensión de imagen	Solicitar identificación de objeto, texto visible, marca, categoría, unidad y atributos candidatos. Tratar la salida como no confiable.	NVIDIA VLM vía API compatible con OpenAI
AGE-03	Salida estructurada	Exigir JSON conforme a schema, confidence por campo y fuentes visuales; validar y reparar solo dentro de límites. Si falla, volver a carga manual.	Pydantic/JSON Schema; adapter de modelos
AGE-04	Búsqueda de coincidencias	Primero usar SKU/código/nombre exacto y filtros SQL; usar embeddings para similitud semántica o visual. PGVector no reemplaza consultas relacionales.	PostgreSQL + pgvector; HNSW solo tras medir volumen
AGE-05	Sugerencia de producto	Proponer nuevo producto o actualización de uno existente; mostrar diferencias, duplicados y nivel de confianza.	AgentSuggestion; GraphQL

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
AGE-06	Herramientas seguras	Exponer search_products, get_product, create_draft y propose_stock_movement. Las herramientas de escritura requieren scope de tenant y confirmación.	Capa de servicios Django; MCP Python SDK opcional
AGE-07	Human-in-the-loop	El usuario valida/edita cada parámetro y confirma. El agente nunca descuenta ni incrementa stock directamente.	Mutation confirmAgentSuggestion; AuditEvent
AGE-08	Orquestación asíncrona	Ejecutar análisis fuera del request web, reportar progreso y permitir reintento/cancelación. Límites por usuario y tamaño.	Celery queue agent; Redis; PostgreSQL AgentRun
AGE-09	Evaluación y versionado	Versionar prompt, modelo, schema y parámetros; mantener set de imágenes de prueba y métricas de exactitud/corrección humana antes de cambiar modelo.	ModelEvaluation; CI jobs; almacenamiento R2 privado
AGE-10	Costos y observabilidad	Registrar latencia, tokens/unidades, fallos y costo estimado sin almacenar prompts sensibles en logs. Cupos y circuit breaker.	Sentry/metrics; provider adapter; feature flags
AGE-11	Privacidad y aislamiento	Separar embeddings y resultados por tenant; filtrar antes de búsqueda vectorial; política de retención y eliminación de fotos.	PostgreSQL RLS opcional + checks de servicio; R2 lifecycle
AGE-12	Portabilidad del modelo	No fijar el dominio a un model_id. Seleccionar VLM de NVIDIA mediante prueba con imágenes reales, español, JSON válido, latencia y costo. En MVP usar API alojada, no un NIM GPU en la VPS general.	NVIDIA API Catalog/NIM; adapter OpenAI-compatible

7. Capacidades transversales

Estas capacidades comienzan en la etapa 1 y se reutilizan en todas las demás. No deben planificarse como una etapa final de infraestructura.

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
TRA-01	Cuenta propia	Registro por email, verificación, inicio/cierre, recuperación y cambio de contraseña. Web: cookie HttpOnly/Secure + CSRF. Mobile: token revocable en almacenamiento seguro. Proteger contra enumeración.	Django auth + django-allauth headless; Expo SecureStore
TRA-02	Google y LinkedIn	Google como login social principal. LinkedIn mediante OIDC actual, habilitando el producto correspondiente; vinculación segura de cuentas por email verificado y confirmación.	django-allauth socialaccount; Google OIDC; LinkedIn OIDC

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
TRA-03	Tenancy y roles	Organisation/Membership; roles owner, operator y support/admin. Compradores usan enlaces públicos y no requieren cuenta en MVP.	Django permissions; filtros obligatorios por tenant
TRA-04	Contrato GraphQL	Schema tipado, queries paginadas, mutations idempotentes, errores con códigos de dominio y límites de complejidad/profundidad.	Strawberry GraphQL Django; GraphQL Code Generator
TRA-05	Endpoints no GraphQL	Mantener callbacks OAuth, webhooks, health/readiness, Turnstile y firma de uploads como endpoints HTTP estrechos y autenticados según el caso.	Django views/ASGI; Nginx routing
TRA-06	Web y landing	Landing indexable y app autenticada en un proyecto React con separación de rutas, bundles y políticas de caché.	Next.js App Router
TRA-07	Mobile	App React Native con cámara, deep links, almacenamiento seguro de sesión y publicación Android/iOS. Compartir tipos y lógica, no asumir reutilización total de UI web.	Expo React Native; EAS Build opcional
TRA-08	Caché cliente	Cache normalizada por ID/tenant, políticas por campo, paginación y actualización explícita tras mutations. No persistir datos sensibles indefinidamente.	Apollo Client; persistencia selectiva en mobile
TRA-09	Tareas asíncronas	Colas separadas default, email, imports y agent; retries con backoff, timeouts, dead-letter operativo y tareas idempotentes.	Celery worker + Redis broker; Celery Beat
TRA-10	Correo	Plantillas transaccionales para cuenta, venta, pago, despacho y ticket; procesar webhooks de entrega/rebote.	Brevo API + webhooks
TRA-11	Almacenamiento	Buckets/prefijos separados por ambiente; media privada por defecto; URLs firmadas cortas, CORS limitado y lifecycle.	Cloudflare R2 S3-compatible
TRA-12	Seguridad pública	Turnstile en registro, contacto y formularios públicos de riesgo; validación obligatoria server-side. Complementar con rate limit, CSP, CSRF/CORS y validación de archivos.	Cloudflare Turnstile/WAF; Nginx limit_req; Django
TRA-13	Auditoría	Registrar actor, tenant, acción, objeto, request/correlation ID y resultado en operaciones críticas. No guardar secretos ni payloads completos sensibles.	AuditEvent; JSON logging
TRA-14	Analítica	Definir taxonomía de eventos y embudo; consentimiento antes de analítica no esencial. Web usa GA4; app usa Google Analytics for Firebase.	GA4 Google tag + Firebase Analytics SDK
TRA-15	Observabilidad	Errores backend/web/mobile, performance y trazas con release/environment; healthchecks y monitor externo de disponibilidad.	Sentry SaaS recomendado; health endpoints

ID	Capacidad	Comportamiento y reglas ejecutables	Herramienta / componente
TRA-16	Accesibilidad y UX	Lenguaje directo, acciones reversibles, estados visibles, objetivos táctiles grandes, WCAG 2.2 AA en web y pruebas con usuarios no nativos digitales.	Design system accesible; axe/Playwright
TRA-17	Pruebas	Unitarias de dominio, integración con PostgreSQL/Redis, contract/schema GraphQL, E2E web y smoke/E2E mobile. Probar webhooks repetidos y concurrencia de stock.	pytest; factory_boy; Playwright; Maestro
TRA-18	CI/CD	Lint, tests, build, scan, imágenes versionadas, migración controlada y despliegue por ambiente. Rollback a imagen previa; nunca tags latest como única referencia.	GitHub Actions + GHCR + Docker Compose
TRA-19	Configuración/ secrets	Configuración por ambiente, secretos fuera de Git y rotación. Producción requiere DEBUG=false, hosts/orígenes explícitos y credenciales distintas.	GitHub Environments/Secrets; SOPS+age recomendado
TRA-20	Backups/ recuperación	Backup off-site de PostgreSQL, configuración y claves necesarias; retención, cifrado y prueba periódica de restauración. R2 no reemplaza backup de DB.	pg_dump + restic/rclone a R2; evolución a pgBackRest/WAL-G
TRA-21	Soporte interno	Administrar usuarios, conexiones de pago, pedidos atascados, reconciliaciones y tickets con permisos y auditoría.	Django Admin endurecido + vistas operativas

8. Arquitectura lógica propuesta

Corrección terminológica: Nginx actúa como reverse proxy de entrada, no como “Nginx de salida”. El flujo esperado es cliente → Cloudflare → Nginx → frontend/backend. El tráfico saliente hacia Brevo, Mercado Pago, NVIDIA u OAuth se origina desde los servicios internos.

Capa	Componentes	Responsabilidad / límites
Clientes	Next.js web/landing; Expo React Native; páginas públicas de compra	UI, validación temprana, caché cliente y telemetría. Nunca fuente de verdad de stock/pago.
Edge	Cloudflare DNS/CDN/WAF/Turnstile	TLS edge, protección, caché pública y desafío. R2 permanece separado como object storage.
Entrada VPS	Nginx	TLS de origen, routing, límites de tamaño/tiempo, rate limit básico y headers de seguridad.
Aplicación	Django ASGI + Strawberry GraphQL + endpoints HTTP específicos	Autorización, reglas de negocio, transacciones, webhooks y firma de uploads.

Capa	Componentes	Responsabilidad / límites
Asíncrono	Celery worker(s) + Celery Beat + Redis	Emails, import/export, expiraciones, recordatorios, conciliación y agente. Redis no guarda estado de negocio durable.
Datos	PostgreSQL + pgvector; Redis; Cloudflare R2	Relacional/auditoría/vectores; caché/broker; media/artefactos. Backups off-site.
Integraciones	Brevo, proveedor de pagos, Google/LinkedIn, NVIDIA, GA4/Firebase, Sentry	Adaptadores aislados, credenciales por ambiente, timeouts y circuit breakers.
Entrega	GitHub Actions + GHCR + Docker Compose	Build una vez, promover imagen versionada, migrar y desplegar con rollback.

8.1 Servicios Docker Compose

Servicio	Desde	Etapas	Notas
nginx	Imagen oficial fijada	1	Único puerto público del stack de aplicación; DB y Redis no se publican.
web	Next.js	1	Landing y app web; build separado por ambiente, sin secretos de servidor expuestos al browser.
backend	Django ASGI	1	GraphQL, auth/callbacks, webhooks y healthchecks.
postgres	PostgreSQL + pgvector	1	Volumen exclusivo por ambiente; backups y migraciones.
redis	Redis	1	Caché/broker; auth/red privada, política maxmemory definida.
celery_worker	Misma imagen backend	1	Colas configurables; puede escalar por proceso sin cambiar arquitectura.
celery_beat	Misma imagen backend	2	Instancia única; agenda durable en PostgreSQL.
agent_worker	Misma imagen backend o imagen dedicada	4	Cola con límites de concurrencia y timeouts externos.
mcp_server	SDK oficial MCP	4 · opcional	Solo si habrá hosts/agentes distintos; no exponer públicamente sin auth y scopes.

8.2 Separación Dev / Producción

- `compose.yaml` como base; `compose.dev.yaml` y `compose.prod.yaml` como overrides explícitos. Validar el resultado con `docker compose config` en CI.
- Nombres de proyecto distintos; redes, volúmenes, dominios, secretos, bases, OAuth callbacks, buckets/prefijos, Brevo y credenciales de pago separados.
- Producción con imágenes inmutables desde GHCR; Dev puede usar bind mounts. No construir código de producción directamente en la VPS.
- Si comparten VPS temporalmente, imponer límites CPU/RAM, firewall, backups separados y acceso a Dev por VPN/allowlist. La opción correcta al crecer es otra VPS para Dev/staging.

9. Herramientas faltantes: opciones y recomendación

Se incluyen solo decisiones que el planteamiento inicial no cerraba o describía de forma ambigua.

Decisión	Recomendada	Alternativa	Razón
Servidor GraphQL	Strawberry GraphQL Django	Graphene-Django	Strawberry: typing moderno, schema explícito y mejor transferencia a un agente. Graphene es viable si el equipo ya lo domina.
Autenticación	django-allauth headless	Keycloak	Allauth cubre cuenta propia y social con menos operación. Keycloak agrega un servicio pesado sin una necesidad IAM clara en este MVP.
Web React	Next.js App Router	React + Vite SPA	Next.js resuelve landing indexable y app en un proyecto. Vite es más simple, pero obliga a resolver SEO/SSR de la landing por otra vía.
Mobile	Expo React Native	React Native bare	Expo reduce build, permisos, cámara, deep links y releases. Bare solo si una dependencia nativa crítica no es compatible.
Cliente/caché GraphQL	Apollo Client	urql + Graphcache	Apollo ofrece caché normalizada y ecosistema más conocido por agentes. Requiere políticas explícitas; no equivale a offline automático.

Decisión	Recomendada	Alternativa	Razón
Pagos Chile	Mercado Pago Checkout Pro + OAuth por vendedor	Transbank Webpay Plus por vendedor	Mercado Pago documenta marketplace con access token de cada vendedor y webhooks. Transbank exige afiliación/credenciales del comercio; útil como segundo adapter.
PDF de etiqueta	WeasyPrint	ReportLab	HTML/CSS facilita iterar etiquetas. ReportLab da control bajo nivel, pero eleva costo de maquetación.
Errores/ performance	Sentry SaaS	GlitchTip self-hosted	El monitor no debería caer con la misma VPS. Autoalojarlo agrega carga y operación al MVP.
Registro de imágenes	GitHub Container Registry	Docker Hub	GHCR se integra con Actions y permisos del repositorio; reduce otra cuenta/credencial.
Push móvil	Expo Notifications	FCM/APNs directo	Expo simplifica el MVP. Integración directa se reserva para necesidades avanzadas o salida de Expo.
Backups MVP	pg_dump cifrado + restic/rclone a R2	pgBackRest o WAL-G	La primera opción es operable con bajo volumen y debe incluir prueba de restore. Cambiar si se exige recuperación a un punto en el tiempo.
E2E mobile	Maestro	Detox	Maestro es más simple para journeys principales. Detox ofrece control profundo con mayor costo de setup.

Decisión de pago: Para el MVP, Tenda debería conectar la cuenta de pago de cada vendedor y evitar recibir fondos en una cuenta propia. Cobrar y repartir dinero como plataforma amplía obligaciones de conciliación, devoluciones, fraude, soporte y cumplimiento; no es un detalle de API.

10. Resumen de herramientas tecnológicas

Categoría	Herramienta	Uso	Ubicación	Etap a
Backend	Django	Reglas de dominio, admin, auth, HTTP/ASGI	Contenedor backend	1
API	Strawberry GraphQL Django	Schema GraphQL tipado	Paquete backend	1
Contrato TS	GraphQL Code Generator	Tipos/hooks compartidos web/mobile	CI / workspace frontend	1
Datos	PostgreSQL	Sistema relacional y transaccional	Contenedor con volumen	1

Categoría	Herramienta	Uso	Ubicación	Etap a
Vector	pgvector	Embeddings y similitud	Extensión PostgreSQL	4
Caché/broker	Redis	Broker Celery, caché, rate/locks auxiliares	Contenedor privado	1
Jobs	Celery Worker + Beat	Tareas, reintentos, expiraciones y agenda	Contenedores internos	1/2
Web	Next.js + React	Landing, app y páginas públicas	Contenedor web	1
Mobile	Expo React Native	Android/iOS, cámara y deep links	Build móvil	1
Caché front	Apollo Client	Caché GraphQL normalizada	Web/mobile	1
Formularios	React Hook Form + Zod	Validación accesible y tipada	Web/mobile cuando aplique	1
Identidad	Django auth + django-allauth	Cuenta propia, Google y LinkedIn OIDC	Backend	1
Object storage	Cloudflare R2 + boto3	Fotos, comprobantes, exports, etiquetas	Servicio externo	1
Anti-bot	Cloudflare Turnstile	Registro/contacto/endpoints públicos	Edge + validación backend	1
Edge	Cloudflare DNS/CDN/WAF	DNS, CDN, TLS y protección	Servicio externo	1
Proxy	Nginx	Reverse proxy, routing y límites	Contenedor público	1
Correo	Brevo API/webhooks	Email transaccional y estado de entrega	Servicio externo	1
Pago	Mercado Pago Checkout Pro/OAuth	Pago online por vendedor	Servicio externo/adaptador	2
Pago alternativo	Transbank Webpay Plus	Segundo proveedor futuro	Adaptador opcional	Post-MVP
PDF	WeasyPrint	Etiqueta interna de despacho	Paquete backend/worker	2
IA	NVIDIA API Catalog / NIM API	VLM/LLM alojado, tool calling	Servicio externo	4
Agente	MCP Python SDK	Frontera estándar de tools	Servicio opcional	4
Analítica web	Google Analytics 4	Embudo web	Next.js	1
Analítica app	Google Analytics for Firebase	Eventos mobile	Expo/native SDK	1
Observabilidad	Sentry + logs JSON	Errores, traces y releases	SaaS + apps	1
CI/CD	GitHub Actions	Tests, build, scan, deploy	GitHub	1
Registry	GHCRC	Imágenes versionadas	GitHub Packages	1
Runtime	Docker Compose	Stacks Dev/Prod	VPS	1
Pruebas	pytest + Playwright + Maestro	Backend, web y mobile E2E	CI	1
Backups	pg_dump + restic/rc1one	Backup cifrado off-site y restore	VPS → R2	1

Categoría	Herramienta	Uso	Ubicación	Etap a
Secrets	GitHub Environments + SOPS/age	Separación y rotación de secretos	CI/VPS	1

11. Decisiones pendientes y criterios de aceptación

Tema	Decisión	Aceptación mínima
Método de pago MVP	¿Mercado Pago por OAuth del vendedor? ¿Tenda cobra comisión?	Pago de prueba end-to-end; webhook repetido; devolución; desconexión del vendedor; no custodia de fondos por Tenda.
Campos de producto	¿SKU/variantes, costo, unidad, categoría y stock mínimo obligatorios?	Schema acordado y migrable; alta manual en menos de 60 s en prueba con usuarios.
Reserva	Duración y comportamiento para pago manual/online.	Prueba concurrente de última unidad; vencimiento libera exactamente una vez.
Despacho	Cadencia de confirmación, recordatorios y autocierre.	Reglas configurables en DB; ningún timer depende solo de Redis.
Transportista	¿Etiqueta interna o integración real con carrier?	Si es interna, lenguaje/UI no promete guía o tracking del carrier.
Facturación	¿Solo datos de comprador o emisión de DTE/SII?	No usar “facturación” como promesa funcional sin proveedor/alcance tributario.
Offline mobile	¿Se exige operar sin conexión o solo tolerar cortes?	Si se exige, definir cola local, conflictos y cifrado; Apollo cache por sí solo no basta.
Modelo NVIDIA	VLM final, embedding y cuotas.	Benchmark con imágenes reales en español, JSON válido, p95 de latencia, costo y tasa de corrección humana.
MCP	¿Habrá más de un host/agente consumidor?	Si no, usar capa de servicios directa y posponer MCP para evitar una frontera sin consumidor.
SLA/ recuperación	RPO/RTO y retención.	Restore probado y documentado; monitor externo detecta caída total de VPS.

12. Reglas para el agente de desarrollo

- Implementar por módulos de dominio (accounts, inventory, sales, shipping, notifications, integrations, agent), no por capas genéricas gigantes.
- Toda mutation crítica debe delegar en un servicio de dominio probado; el resolver GraphQL no contiene reglas transaccionales.

- Usar UUID/ULID u otro identificador no predecible en URLs públicas; además usar tokens firmados y expirables donde corresponda.
- Nunca publicar PostgreSQL o Redis en el host. Nginx/Cloudflare son la entrada pública; R2 usa credenciales de mínimo privilegio.
- Persistir primero y ejecutar efectos externos mediante outbox/Celery. Webhooks, tareas y mutations con idempotencia verificable.
- Fijar versiones soportadas en lockfiles e imágenes; no copiar versiones de este documento como tags flotantes ni usar latest en Producción.
- Cada capacidad ID debe convertirse en épica/historias y pruebas; mantener trazabilidad BPMN → capacidad → mutation/query/job → prueba.
- No implementar vistas de la etapa siguiente antes de acordar el modelo de datos, estados e invariantes de la etapa actual.

13. Referencias técnicas consultadas

django-allauth Headless	django-allauth · proveedor Google
django-allauth · LinkedIn mediante OIDC	Strawberry GraphQL Django
Cloudflare R2 · URLs firmadas	Cloudflare Turnstile · validación server-side
Celery · Redis	Next.js
Expo React Native	Apollo Client · caché
Mercado Pago · marketplace	Mercado Pago · webhooks
Transbank · Webpay Plus API	Brevo · email transaccional
Brevo · webhooks transaccionales	NVIDIA NIM · API compatible con OpenAI
NVIDIA NIM · API de modelos vision-language	pgvector
Model Context Protocol · arquitectura	Google Analytics · web y app
GitHub Actions · publicar imágenes en GHCR	Docker Compose · múltiples archivos